

COS214 Practical 5

Ethan Van Eyden (u25260244)

Shaun Mautsa (u24693228)

Mueez Gani (u24983439)

Table of Contents

Section 1 – Collaboration.....	1
GitHub link	2
Contribution statement	2
Section 2 – System description.....	3
Section 3 – Class diagram	4
Section 4 – Pattern mapping.....	5
Command	6
Mediator.....	6
State	6
Adapter	7
Factory Method.....	7
Facade	8
Section 5 – Pattern justification	9
State	10
Factory Method.....	10
Section 6 – Design rationale	10
Architecture overview	11
Ownership and destruction policy	11

Pattern interaction	12
Invalid-operation handling	12
Section 7 – Behavioral diagrams	12
Section 8 – Runtime evidence.....	13
GDB report	14
Valgrind report	14

Section 1 – Collaboration

GitHub link

[ethanvaneyden/COS214-prac5](https://github.com/ethanvaneyden/COS214-prac5)

Contribution statement

Ethan – I coded the mediator (not the colleagues), the facade, everything involving the incident and its states and the incident registry. I also built the interactive demo. Furthermore, I was also responsible for the makefile and docker and wrote the valgrind report. On the report I was responsible for the UML class diagram, the design rationale, ownership model and the system description.

Shaun – I implemented the Command and Adapter patterns for CampusGuard. This included the Command interface, the CommandInvoker (which executes commands and records a history), and the three concrete commands — DispatchCommand, ResolveCommand, and CancelCommand — each acting as a thin wrapper around a real Incident state transition. For the Adapter, I built the AlarmSystem target interface, the LegacyAlarmSystem adaptee simulating the campus's existing siren hardware with its incompatible zone/mode interface, and the LegacyAlarmAdapter that translates between the two and converts hardware failures into exceptions rather than swallowing them. I wrote and verified the `testCommand()` and `testAdaptor()` functions in `main.cpp`, including the illegal-transition and unknown-zone failure cases required by the brief. On the report, I was responsible for the pattern mapping and justification for Command and Adapter, and I produced both required UML sequence diagrams — the dispatch flow (Command/State/Mediator) and the evacuate-building flow (Facade/Adapter) — tracing each message against the actual implementation.

Mueez – I implemented the Factory Method section of CampusGuard, including the ResponseUnit hierarchy, the four concrete response units (SecurityUnit, MedicalUnit,

MaintenanceUnit, and CommunicationsUnit), and their corresponding concrete factories. I also integrated the units with the ControlRoom mediator through event callback methods and helped test the interaction between the Factory Method, Mediator, and Adapter patterns. During integration I used GDB to investigate and fix a segmentation fault caused by accessing a unique_ptr after ownership had been transferred with std::move. I also contributed to the final integrated demo and static testing.

Collaboration reflection

The team divided work along pattern boundaries — Ethan on Mediator, State and Facade, Mueez on Factory Method and the response units, and Shaun on Command and Adapter. We used GitHub Issues to track work and pull requests to merge each member's branch into main, which kept changes isolated and gave us a clear record of who worked on what. The main challenge was the integration seams between our areas — the ResponseUnit interface between Ethan's mediator and Mueez's colleagues, and the AlarmSystem interface between Ethan's facade and Shaun's adapter. We did not stub these early, so work stalled whenever a teammate's header was not yet pushed, and late interface changes rippled through ControlRoom.cpp. The static tests in main.cpp acted as our integration checkpoint — once all three members' tests passed, the seams were sound. Overall, the team communicated well and resolved interface disagreements quickly. If we repeated the project, we would freeze the shared interfaces in the first session, so nobody was ever blocked waiting on another's headers.

Section 2 – System description

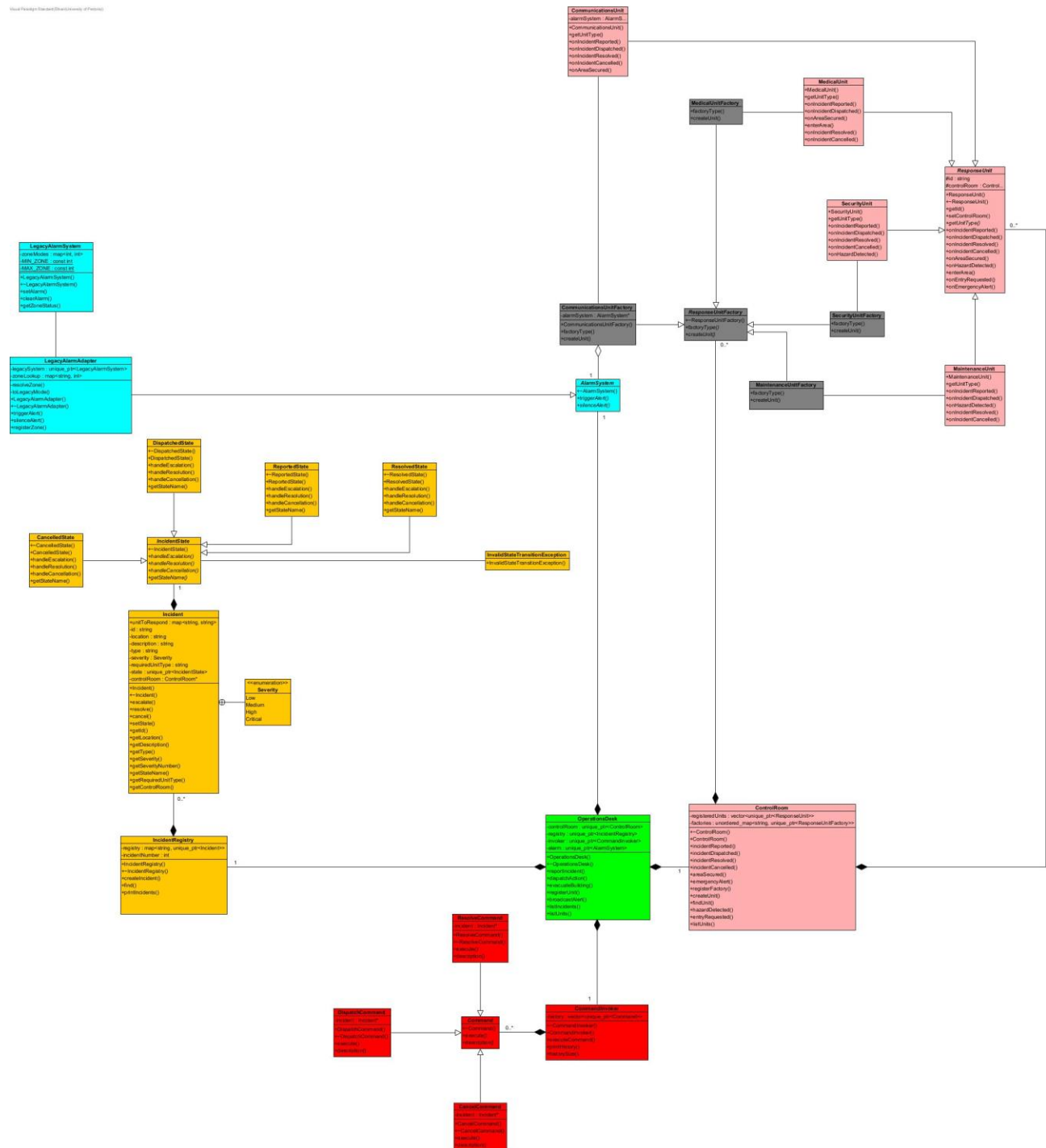
CampusGuard is a command-line emergency-response coordination platform for a university campus. It allows an operator to register incidents, dispatch the appropriate response teams, and coordinate multi-step emergency workflows from a single interface.

The system supports a range of incident types — including medical emergencies, theft, intruders, fire, and gas leaks — and coordinates four response units: Security, Medical, Maintenance and Communications. A Facade provides the operator with a single-entry point, while a Mediator coordinates the response units so that they never depend directly on one another. Each incident follows a state-driven lifecycle, and operator actions are represented as Command objects.

The system's responsibilities are:

- Registering incidents and tracking their lifecycle state
- Dispatching response units to incidents
- Coordinating between response units when a colleague raises an issue (e.g. a hazard discovered on scene)
- Providing a single operator interface that hides subsystem complexity
- Integrating with the legacy campus alarm system via an adapter

Visual Perception Specialist (University of Portland)



Section 4 – Pattern mapping

Command

1. Command (Command) - Provides the abstract interface (execute(), description()) that every operator action must implement.
2. DispatchCommand, ResolveCommand, CancelCommand (ConcreteCommand)- Each wrap one real operator action and calls exactly one method on its receiver: Incident::escalate(), Incident::resolve () or Incident::cancel ().
3. Incident (Receiver) -Receives the call and delegates to its current IncidentState, which decides whether the transition is legal and performs it.
4. CommandInvoker (Invoker) - Executes the command it's given and records it in a history; it never creates commands and doesn't know which concrete type it's running.
5. OperationDesk (Client) - Builds the appropriate concrete Command for the operator's requested action, sets its receiver (the target Incident) and passed it to the CommandInvoker for execution.

Mediator

1. ControlDesk (Mediator) – The control desk allows the different response units to communicate with each other and send updates to each other when they respond to an incident.
2. ResponseUnit (Colleague) – Provides the abstract interface of what behaviour a response unit should support.
3. SecurityUnit, MedicalUnit, MaintenanceUnit, CommunicationsUnit (ConcreteColleagues) – These are the concrete response units coordinated by the ControlRoom. They react to mediator callbacks and report important events back through the mediator instead of communicating directly with each other.

State

1. Incident (Context) – Represents an incident that needs to be tended to.

2. IncidentState (State) – Provides the abstract interface that defines behaviour of what states may do
3. ReportedState, DispatchedState, ResolvedState, CancelledState (ConcreteState) – Provides the actual implementations and behaviours of each state and the transition rules

Adapter

1. AlarmSystem (Target) - Provides the interface CampusGuard itself wants to program against: raising and silencing an alert by location and severity level
2. LegacyAlarmSystem (Adaptee) - The existing campus siren hardware. Its interface predates CampusGuard and can't be changed. It only understands integer zone codes, integer modes and reports success/ failure via return codes rather than exceptions.
3. LegacyAlarmAdapter (Adapter) - Implements AlarmSystem and translates between the two interfaces: converts a location name into a hardware zone code and a severity level into a legacy mode then forwards the call to LegacyAlarmSystem and converts its failure codes into thrown exceptions
4. OperationDesk(Client) - Owns the AlarmSystem (constructed as a LegacyAlarmAdapter) and calls it purely through the AlarmSystem interface, with no knowledge that a legacy system sits behind it. Builds the appropriate concrete Command for the operator's requested action, sets its receiver (the target Incident) and passes it to the CommandInvoker for execution

Factory Method

1. ResponseUnitFactory (Creator) – Provides the abstract factory interface through createUnit(id) for creating response units.
2. SecurityUnitFactory, MedicalUnitFactory, MaintenanceUnitFactory, CommunicationsUnitFactory (ConcreteCreators) – Each factory creates one specific type of response unit and returns it through the common ResponseUnit interface.

3. ResponseUnit (Product) – Provides the common interface for all response units created by the factories.
4. SecurityUnit, MedicalUnit, MaintenanceUnit, CommunicationsUnit (ConcreteProducts) – The actual response-unit objects created by their corresponding concrete factories.
5. ControlDesk (Client) – Selects the correct factory based on the requested unit type, calls createUnit(id), connects the created unit to the mediator and stores the unit.

Facade

1. OperationsDesk (Facade) – Provides a simplified high-level interface for the operator and hides the interactions between the different CampusGuard subsystems.
2. IncidentRegistry (Subsystem) – Creates, stores and retrieves incidents used by the facade.
3. ControlRoom (Subsystem) – Manages response units and coordinates communication between them.
4. CommandInvoker and Commands (Subsystem) – Carry out operator actions such as dispatching, resolving and cancelling incidents.
5. AlarmSystem / LegacyAlarmAdapter (Subsystem) – Allows the facade to activate or silence the campus alarm system through the Adapter interface.
6. evacuateBuilding() (Facade Operation) – Coordinates multiple subsystem operations in one call by triggering the alarm, broadcasting an emergency alert, registering an evacuation incident and dispatching the required response unit.

Section 5 – Pattern justification

State

State was a very natural choice for this software. Incidents naturally follow states as they go from being reported to being resolved. The state pattern allowed us to enforce strict transition rules (can't cancel a resolved incident) and allowed the software to delegate the logic of when a unit can be dispatched to the state of the incident.

Factory Method

Factory Method was also a very natural choice for this software. It allows creation of response units at run time easily should there be no team that can respond to the incident. It also helps the software follow the open closed principle better since adding a new response unit type is much easier. You don't need if else logic to decide what unit to create, only add one entry to an unordered map that maps the unit type string to the appropriate factory.

Section 6 – Design rationale

Architecture overview

CampusGuard is built around single operator facing façade (OperationsDesk). The façade owns 4 subsystems: An incident registry that stores the incidents, a ControlRoom that mediates between response units, a ComandInvoker that executes operator actions, and an AlarmSystem adapter that integrates the legacy campus alarm. Operator input flows through the façade into command objects that call into incidents whose state pattern then decides the next transition. On a successful transition, the ControlRoom gets notified which routes the event to all registered response units. Response units do not hold references to each other and all coordination passes through the mediator.

Ownership and destruction policy

All ownership is done using unique pointers which automatically free memory as soon as the object goes out of scope thus removing the need to code any destructors and makes memory leaks impossible which makes the code very robust.

Owner	Owned objects	Method
OperationsDesk	ControlRoom, IncidentRegistry, CommandInvoker, AlarmSystem	Unique_ptr
IncidentRegistry	All incident objects	Map
ControlRoom	All ResponseUnit objects	Vector
ControlRoom	All ResponseUnitFactory objects	Unordered map
Incident	Its current IncidentState	Unique_ptr
CommandInvoker	All executed Command objects	Vector

Non-owning references:

Observer	Points to	Why non owning
ResponseUnit	ControlRoom	The mediator outlives its colleagues

Incident	ControlRoom	Incidents are owned by the registry. They observe the mediator but don't own it
Command	Incident	Commands are transient and incidents outlive them

Pattern interaction

The six patterns all rely on each other and interconnect and work together in one execution path. An operator enters the dispatch action into the CLI and then it gets forwarded to the façade which decides what to do with it. The OperationsDesk constructs the command and passes it to the CommandInvoker. The command calls escalate on the incident which then delegates the action to the current state. The state performs the transition and then notifies the Mediator (ControlRoom). The mediator fans the event out to all registered ResponseUnits who then decide what to do with the information. When a unit discovers something on the scene such as a hazard or an entry request, it renotifies the mediator, which then triggers further coordination. The factory method provides ResponseUnits at runtime, so the mediator never needs to know the concrete classes. The adaptor sits behind the facades evacuate building workflow to translate the legacy alarm systems incompatible interface.

Invalid-operation handling

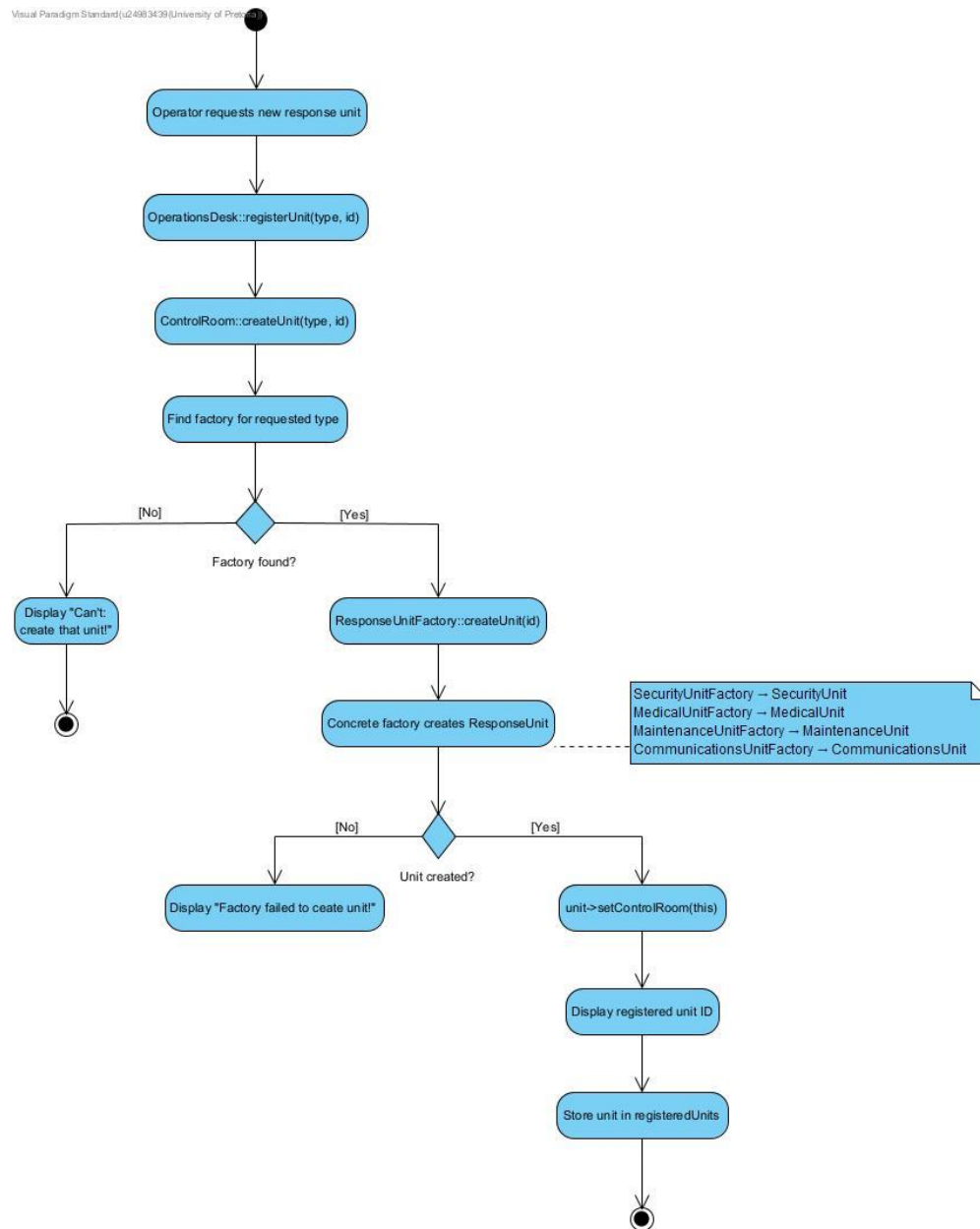
Invalid state transitions are handled with an exception class (InvalidStateTransitionException). Illegal state transitions such as dispatching to a resolved incident, resolving an incident that was never dispatched throw this exception. OperationsDesk catches this and prints the rejection message.

```

sequenceDiagram
    actor Main
    participant desk as desk: OperationDesk
    participant alarm as alarm: LegacyAlarmAdapter
    participant LegacyAlarmSystem as LegacyAlarmSystem
    participant ControlRoom as ControlRoom
    participant ResponseUnit as ResponseUnit
    participant IncidentRegistry as IncidentRegistry
    participant Incident as Incident
    participant CommandCenter as CommandCenter

    Main->>desk: 1: evacuateBuilding(buildingId: string)
    activate desk
    desk->>alarm: 1.1: triggerAlert(buildingId)
    activate alarm
    alarm->>LegacyAlarmSystem: 1.1.1: setAlarm(zone, mode) : int
    activate LegacyAlarmSystem
    LegacyAlarmSystem-->>alarm: 1.1.2: 0
    deactivate LegacyAlarmSystem
    alarm->>ControlRoom: 1.2: emergencyAlert(message:string)
    activate ControlRoom
    ControlRoom->>ResponseUnit: loop  
[for each unit in registeredUnits]  
1.2.1: onEmergencyAlert(message)
    activate ResponseUnit
    deactivate ResponseUnit
    ControlRoom-->>desk: 1.3: reportIncident(type, location, description, severity) : string
    deactivate ControlRoom
    activate desk
    desk->>IncidentRegistry: 1.4: createIncident(location, description, type, severity, room)
    activate IncidentRegistry
    IncidentRegistry->>Incident: 1.4.1: Incident(location, description, type, severity, room)
    activate Incident
    deactivate Incident
    IncidentRegistry-->>desk: 1.4.2.1: IncidentReported(id, location, severity)
    deactivate IncidentRegistry
    activate desk
    desk->>ControlRoom: 1.4.2.1: onIncidentReported(id, location, severity)
    activate ControlRoom
    ControlRoom->>ResponseUnit: 1.4.2.1: onIncidentReported(id, location, severity)
    activate ResponseUnit
    deactivate ResponseUnit
    ControlRoom-->>desk: 1.4.2.2: findId() : Incident
    deactivate ControlRoom
    activate desk
    desk->>IncidentRegistry: 1.4.2.2: inc
    activate IncidentRegistry
    deactivate IncidentRegistry
    IncidentRegistry-->>desk: 
    deactivate IncidentRegistry
    desk->>Main: 1.4.2.2: dispatchAction(id, "dispatch")
    deactivate desk
    activate Main
    Main->>CommandCenter: 
    deactivate Main
    Note over CommandCenter: dispatch to unit, Command -> State -> Mediator
    
```

Diagram 3: Factory method activity diagram



Section 8 – Runtime evidence

GDB report

During the guided demo, the program initially crashed with a segmentation fault while registering the first response unit. GDB was used to locate the source of the error.

The program was started in GDB using:

```
gdb ./campusgaurd
run --demo
```

After the crash occurred, the following command was used:

```
bt
```

The backtrace showed that the crash happened inside `ResponseUnit::getId()` and that the object pointer was `this = 0x0`. The call came from `ControlRoom::createUnit()`.

The cause of the error was that the unit unique pointer was being accessed after its ownership had already been transferred using `std::move()`:

```
registeredUnits.push_back(move(unit));
cout << "[Control Room] registered " << unit->getId() << "\n";
```

After `move(unit)`, the local unit pointer becomes empty. Calling `unit->getId()` afterwards therefore caused the segmentation fault.

The problem was fixed by accessing the unit before transferring ownership:

```
cout << "[Control Room] registered "
    << unit->getId() << "\n";
```

```
registeredUnits.push_back(move(unit));
```

After making this change, the project compiled successfully, all static tests passed, and the complete guided demo ran without a segmentation fault. This also confirmed that ownership of response units is correctly transferred to the `ControlRoom` using `std::unique_ptr`.

Valgrind report

```
Units registered:

SEC-01 (security)
MED-01 (medical)
MNT-01 (maintenance)
COM-01 (comms)

[press Enter]

=====
Guided demo complete
=====
Patterns demonstrated in this run:
Facade, Command, State, Mediator,
Factory Method, Adapter

=====
CampusGuard
=====
1. Guided demo (recommended)
2. Free mode (manual console)
3. Run static tests
0. Exit
> 0
Goodbye.
==7==
==7== HEAP SUMMARY:
==7==    in use at exit: 0 bytes in 0 blocks
==7==   total heap usage: 108 allocs, 108 frees, 81,587 bytes allocated
==7==
==7== All heap blocks were freed -- no leaks are possible
==7==
==7== For lists of detected and suppressed errors, rerun with: -s
==7== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

After running the guided demo with docker and valgrind it confirms that all memory is correctly deleted and no memory leaks are present. This means the ownership model worked effectively, and the unique pointers managed memory management completely.

Section 9 – Docker run instructions

The complete Docker configuration and run instructions are reproduced below. These match the files in the repository and were used to produce the runtime evidence in the previous section.

Dockerfile

```
FROM ubuntu:22.04
```

```
RUN apt-get update && apt-get install -y \
```

```
    build-essential \
```

```
    valgrind \
```

```
    gdb \
```

```
    && rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN make
```

```
CMD ["/campusguard"]
```

Docker compose

```
services:
```

```
    campusguard:
```

```
        build: .
```

```
        stdin_open: true
```

```
tty: true
```

Docker run instructions

docker compose up does not attach stdin to a single container by design. To launch and interact with the CampusGuard CLI, use:

```
docker compose run --rm --build campusguard
```

Alternatively, to launch with docker compose up and then attach your terminal:

```
docker compose up --build -d
```

```
docker compose attach campusguard
```